Department AI Assistant

A knowledge assistant for retrieving department-specific information from academic documents.

Problem Statement

Current Challenges

- Department information is distributed across multiple documents (PDFs, notices, manuals, reports, circulars, etc.).
- Students and faculty spend significant time searching for required information.
- Traditional chatbots cannot reliably answer department-specific questions.
- Important information may be outdated or difficult to locate.

Solution

Implement a Retrieval-Augmented Generation (RAG) system that combines document retrieval with a Large Language Model.

RAG approach

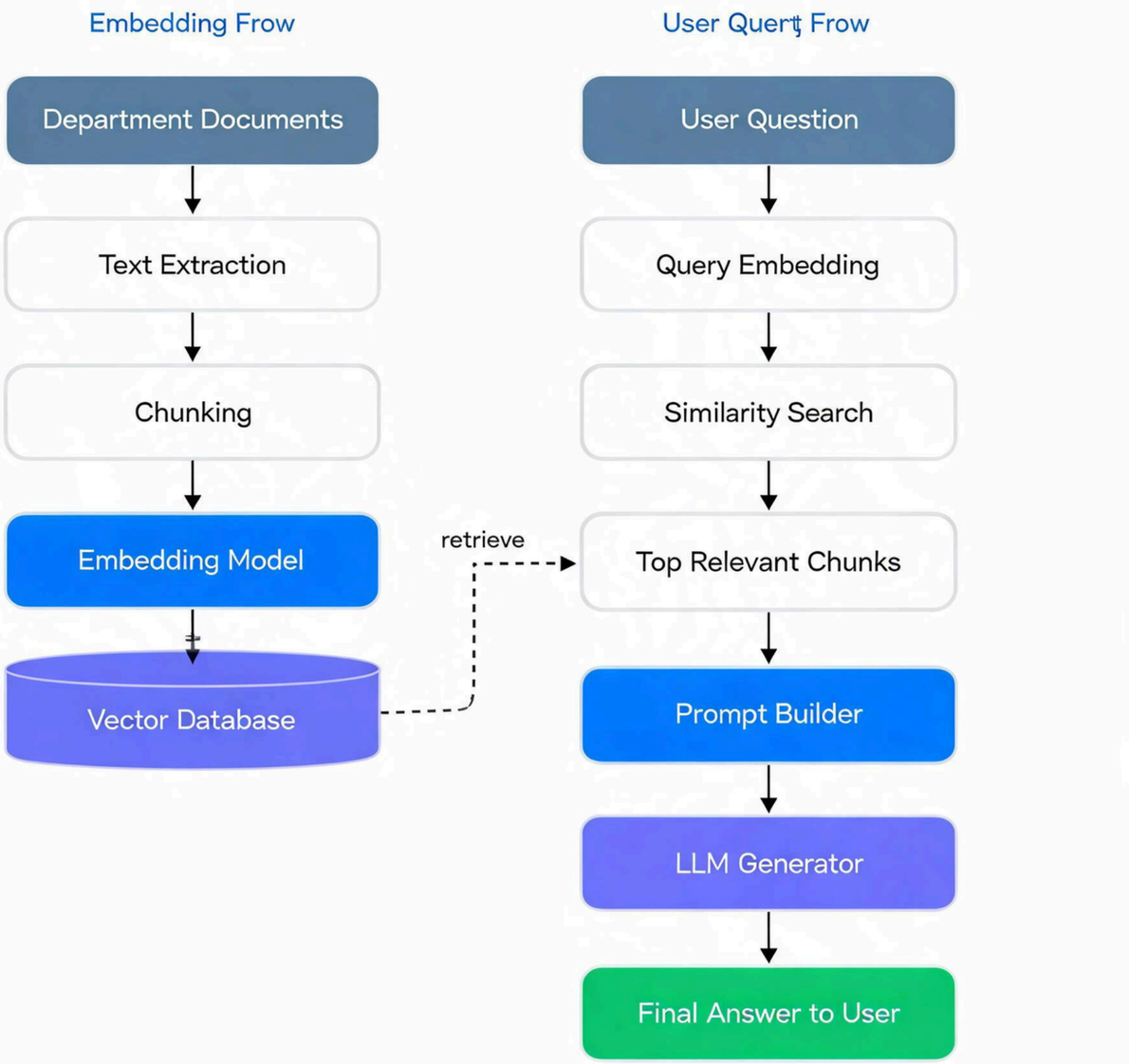
How it Works

- Store department documents in a searchable knowledge base.
- Retrieve the most relevant document sections for each user query.
- Provide the retrieved context to the LLM.
- Generate an accurate, context-aware response.

Expected Outcome

- Faster information access
- Reduced incorrect answers
- Responses based on department document

Architecture



Document processing pipeline

1. Collect documents

Gather department resources such as:

- Notes
- Circulars
- Lab manuals
- Reports
- Academic regulations
- Project guidelines

2. Split into chunks

- Split large documents into smaller meaningful sections.
- Improve retrieval accuracy.
- Keep prompt size manageable.

3. Generate Embeddings

- Convert each chunk into vector representations.
- Capture semantic meaning instead of exact keywords.

4. Store in Vector Database

- Save embeddings with document metadata.
- Enable efficient semantic similarity search.

RAG Query Workflow

Retrieval stage

- User submits a query.
- Convert the query into an embedding.
- Search the vector database.
- Retrieve the Top-K most relevant document chunks.

Generation stage

- Combine the user query with retrieved context.
- Send the complete prompt to the LLM.
- Generate a grounded response based on retrieved informa

System Components

Component	Responsibility
Knowledge Base	Stores department documents
Chunking Module	Splits documents into smaller chunks
Embedding Model	Converts text into vector embeddings
Vector Database	Stores and retrieves embeddings
Retriever	Finds the most relevant document chunks
Prompt Builder	Combines retrieved context with the user query
Large Language Model	Generates the final response

Advantages

Better accuracy

Answers come from department documents rather than only from model memory.

Faster information access

Students and faculty can find rules, formats, and notices quickly.

Easy document updates

New notices can be indexed without retraining the full model.

More trust

Retrieved chunks can be shown as supporting context for the answer.

Thank You